

ICL-BLDC modifications

Training Process



Giuseppe Fiengo

January 7, 2026

Contents

1	Learning Setup and Data Processing	2
1.1	Loading DataSets	2
1.2	Time-Weighted Mean Squared Error Loss	4
1.3	Resource Monitoring and Compute Profiling	5
2	Training Process	7
2.1	Autoregressive Regression Step	7
2.2	Smoothness Regularization via Finite Differences	9
3	Transformer Module Design	11
3.1	Activation Function in the Feed-Forward Network	11
4	Proposed Improvements and Extensions	13
4.1	Mitigating Training–Inference Mismatch	13
4.2	Model-Level Enhancements	14
4.3	Evaluation Strategy and Test-Time Monitoring	14

Chapter 1

Learning Setup and Data Processing

1.1 Loading DataSets

Algorithm 1 Dataset Configuration and Loading

Require: Configuration dictionary $\mathcal{C}$

Ensure: Cached data pools and normalized folder definitions

- 1: Extract split ratios $(r_{\text{train}}, r_{\text{val}}, r_{\text{test}})$
 - 2: **if** $r_{\text{train}} + r_{\text{val}} + r_{\text{test}} \neq 1$ **then**
 - 3: **raise error**
 - 4: Read folder lists $(F_{\text{train}}, F_{\text{val}}, F_{\text{test}})$ from $\mathcal{C}$
 - 5: **if** all folder lists are undefined **then** ▷ Backward compatibility
 - 6: Set $F_{\text{train}} = F_{\text{val}} = F_{\text{test}} = F_{\text{base}}$
 - 7: **else**
 - 8: Fill missing folder lists by reusing available definitions
 - 9: Convert folder lists to sets:
$$S_{\text{train}}, S_{\text{val}}, S_{\text{test}}$$
 - 10: Identify all unique folders:
$$S_{\text{all}} = S_{\text{train}} \cup S_{\text{val}} \cup S_{\text{test}}$$
 - 11: **for** each folder $f \in S_{\text{all}}$ **do**
 - 12: Load all data files from f
 - 13: Cache loaded files in memory
 - 14: Initialize deterministic random generator with fixed seed
 - 15: **return** cached data pools and normalized folder sets
-

Algorithm 2 Dataset Splitting Logic

Require: Cached data pools and folder sets ($S_{\text{train}}, S_{\text{val}}, S_{\text{test}}$)

Ensure: Datasets ($\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$)

- 1: **if** $S_{\text{train}} = S_{\text{val}} = S_{\text{test}}$ **then** ▷ Original behavior
 - 2: Collect all files into a single pool $\mathcal{P}$
 - 3: Shuffle $\mathcal{P}$
 - 4: Split $\mathcal{P}$ into
$$\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$$

using $(r_{\text{train}}, r_{\text{val}}, r_{\text{test}})$
 - 5: **else if** exactly two of the folder sets are equal **then** ▷ Priority-based partial split
 - 6: Assign the distinct folder set entirely to its dataset
 - 7: Merge data from the two equal folder sets into pool $\mathcal{P}$
 - 8: Shuffle $\mathcal{P}$
 - 9: Split $\mathcal{P}$ according to priority:
 - Train keeps r_{train} if involved
 - Validation keeps r_{val} if only val/test are involved
 - 10: **else** ▷ Fully disjoint folders
 - 11: Collect datasets independently:
$$\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$$
 - 12: Shuffle each dataset independently
 - 13: Construct sequence datasets with fixed sequence length
 - 14: **return** ($\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$)
-

1.2 Time-Weighted Mean Squared Error Loss

Let $y_{\text{pred}}, y_{\text{true}} \in \mathbb{R}^{B \times T \times D}$ denote, respectively, the predicted and ground-truth output sequences, where B is the batch size, T the sequence length, and D the output dimension.

Standard Mean Squared Error

The standard mean squared error (MSE) loss over a sequence is defined as

$$\mathcal{L}_{\text{MSE}} = \frac{1}{BTD} \sum_{b=1}^B \sum_{t=1}^T \sum_{d=1}^D \left(y_{\text{pred}}^{(b,t,d)} - y_{\text{true}}^{(b,t,d)} \right)^2. \quad (1.1)$$

This formulation assigns equal importance to all time steps in the sequence. While this choice is simple and stable, it implicitly assumes that prediction errors are equally relevant across the entire temporal horizon.

Time-Weighted MSE Formulation

To explicitly emphasize prediction accuracy at later time steps, a time-weighted variant of the MSE is adopted. First, a per-time-step MSE is computed as

$$\text{MSE}_t^{(b)} = \frac{1}{D} \sum_{d=1}^D \left(y_{\text{pred}}^{(b,t,d)} - y_{\text{true}}^{(b,t,d)} \right)^2. \quad (1.2)$$

A set of non-negative temporal weights $\{w_t\}_{t=1}^T$ is then defined according to one of the following schemes:

$$w_t = \frac{t}{T}, \quad (\text{linear weighting}) \quad (1.3)$$

$$w_t = \exp\left(\alpha \frac{t-1}{T-1}\right), \quad (\text{exponential weighting}) \quad (1.4)$$

where $\alpha > 0$ controls the strength of the exponential weighting.

To ensure scale invariance with respect to the sequence length, the weights are normalized such that

$$\tilde{w}_t = \frac{w_t}{\sum_{k=1}^T w_k}. \quad (1.5)$$

The resulting time-weighted MSE loss is defined as

$$\mathcal{L}_{\text{TW-MSE}} = \frac{1}{B} \sum_{b=1}^B \sum_{t=1}^T \tilde{w}_t \text{MSE}_t^{(b)}. \quad (1.6)$$

Discussion

Compared to the standard MSE, the proposed loss introduces an explicit inductive bias that prioritizes accuracy at later time steps in the sequence. This choice is motivated by scenarios in which early predictions mainly serve as context, while the quality of long-horizon predictions is more critical for evaluation and downstream use.

The main advantage of this formulation is improved alignment with long-horizon performance metrics. However, it may also reduce gradient contributions from early time steps and increase sensitivity to noise at later steps. Consequently, the weighting strategy and the parameter α must be selected with care, and the standard MSE remains a relevant baseline when uniform temporal accuracy or training stability is the primary concern.

1.3 Resource Monitoring and Compute Profiling

To characterize the computational cost of training and to support reproducibility across machines and runs, a lightweight `ResourceMonitor` utility was added. The monitor periodically samples system-, process-, and (when available) GPU-level metrics during training and stores them as a time series. This enables post-hoc analysis of whether a given configuration is compute-bound (GPU utilization saturated) or input/CPU/I/O-bound (low GPU utilization with high CPU, disk, or RAM activity).

Logged quantities

At each sample time, the monitor records:

- **System CPU and RAM:** CPU utilization, total/available memory, and memory percentage.
- **Process statistics:** resident memory (RSS), virtual memory (VMS), number of threads, and process CPU utilization.
- **I/O deltas:** network and disk read/write bytes since monitoring started.
- **GPU statistics (if NVML is available):** GPU utilization, memory usage, temperature, power draw and power limit, SM/memory clock, and (when supported) ECC error counters.

GPU metrics are obtained via NVIDIA’s NVML interface when available; otherwise, the monitor logs NaN values while keeping the same schema, ensuring consistent CSV/DataFrame columns across different hardware setups.

Design choices and parameter rationale

The monitor is designed to be minimally intrusive: sampling is performed in a background daemon thread and uses a fixed sampling interval

$$\Delta t = 5 \text{ s}$$

(`sample_interval_s = 5.0`). This interval was chosen as a compromise between temporal resolution and overhead. Training dynamics (e.g., steady-state GPU utilization, memory growth, thermal or power throttling) typically evolve over timescales of seconds to minutes rather than milliseconds; therefore, a 5-second interval provides sufficient fidelity for diagnosing bottlenecks without significantly perturbing training throughput or generating excessively large logs.

The parameter `device_index = 0` fixes the monitored GPU to the primary device, matching the default single-GPU training configuration. Finally, the switch `enabled` allows disabling monitoring entirely to guarantee that experiments can be run with zero monitoring overhead when desired.

Practical motivation. This profiling step was introduced because the proposed training procedure includes computationally heavier components (e.g., autoregressive unrolling over a window and additional derivative-based regularization). Resource monitoring makes it possible to empirically verify how these design choices impact compute utilization, memory footprint, and I/O behavior, and to distinguish genuine model-side slowdowns from data-loading or system-level bottlenecks.

Chapter 2

Training Process

2.1 Autoregressive Regression Step

Training is performed over mini-batches of input-output windows of fixed length H . Each batch contains

$$u \in \mathbb{R}^{B \times H \times F}, \quad y \in \mathbb{R}^{B \times H \times 1},$$

where B is the batch size and F is the number of input features. The model is trained in an autoregressive (teacher-free) manner: the predicted speed $\hat{\omega}$ is recursively fed back into the input at the subsequent time step, in order to match the conditions encountered at inference time.

New implementation

Let $f_\theta(\cdot)$ denote the Transformer model with parameters θ . Before the recursion, the input channel corresponding to the speed signal (channel index 4 in the implementation) is set to zero, ensuring that the model cannot trivially copy the ground-truth value. The regression procedure then generates predictions sequentially as:

$$\hat{\omega}_0 := 0, \tag{2.1}$$

$$\hat{\omega}_t := f_\theta \left(\tilde{u}_{1:t} \right)_{\text{last}}, \quad t = 1, \dots, H, \tag{2.2}$$

where $\tilde{u}_{1:t}$ denotes the strictly-causal prefix input sequence (from time 1 to t) in which the speed channel at time t has been overwritten with the previous estimate $\hat{\omega}_{t-1}$, and $(\cdot)_{\text{last}}$ indicates that the model output is taken at the last time position. The resulting prediction window is assembled as

$$\hat{y} = [\hat{\omega}_1, \hat{\omega}_2, \dots, \hat{\omega}_H] \in \mathbb{R}^{B \times H \times 1}. \tag{2.3}$$

The primary data-fit term is computed as

$$\mathcal{L}_{\text{fit}} = \mathcal{C}(y, \hat{y}), \tag{2.4}$$

where $\mathcal{C}(\cdot, \cdot)$ is the chosen regression criterion (e.g., MSE or time-weighted MSE).

A key architectural change relative to the earlier implementation is that the regression logic is factored out into a dedicated function `regression()`, controlled through the parameter `regression_mode`. In the default mode (`time_last`) the behavior corresponds to the explicit recursion in (2.2), producing one prediction per time step by repeatedly running the model on progressively longer prefixes.

Old implementation

In the original training loop, the regression step was implemented inline inside the `train` function. Functionally, the autoregressive recursion was the same: the speed channel was zeroed and then overwritten at each step with the previous predicted value, and the prediction sequence $\hat{y}$ was constructed by concatenating last-token predictions across time. The main difference is that the earlier code coupled this regression recursion with the smoothness penalty by concatenating additional signals into the regression loss itself (see Section 2.2), rather than treating smoothness as a separate additive term.

Practical consequence. The new implementation cleanly separates: (i) autoregressive sequence generation (regression), and (ii) objective design (data-fit and smoothness). This makes it possible to compare regression modes and regularization terms without changing the core training loop structure.

2.2 Smoothness Regularization via Finite Differences

In addition to the data-fit loss, a smoothness term is optionally introduced to discourage unrealistic step-to-step variations in the predicted speed trajectory. This is motivated by the fact that physically plausible motor speed dynamics are typically smooth at the sampling rates used in the dataset; therefore, penalizing abrupt variations can improve trajectory realism and reduce high-frequency artifacts.

Let the discrete first difference operator along time be defined as

$$\Delta y_t := y_t - y_{t-1}, \quad t = 2, \dots, H, \quad (2.5)$$

applied independently for each element in the batch. In tensor form, $\Delta y \in \mathbb{R}^{B \times (H-1) \times 1}$.

New smoothness formulation: dynamics-matching penalty

The new training procedure introduces a derivative matching term comparing the first differences of predicted and true trajectories:

$$\Delta y := \text{diff}(y, \text{time}), \quad (2.6)$$

$$\Delta \hat{y} := \text{diff}(\hat{y}, \text{time}). \quad (2.7)$$

A weighted target derivative is then defined as

$$\Delta y_{\text{target}} := w_{\text{dyn}} \odot \Delta y, \quad (2.8)$$

where $\odot$ denotes elementwise multiplication and $w_{\text{dyn}} \in [0, 1]^{B \times (H-1) \times 1}$ is a data-dependent weight computed from the magnitude of the true derivative:

$$w_{\text{dyn}} := \text{clip}\left(\frac{|\Delta y|}{\tau}, 0, 1\right), \quad (2.9)$$

with threshold $\tau > 0$ (fixed in the implementation) and $\text{clip}(\cdot)$ clamping the value into $[0, 1]$.

The resulting smoothness loss is

$$\mathcal{L}_{\text{dyn}} = \|\Delta \hat{y} - \Delta y_{\text{target}}\|_2^2 = \frac{1}{B(H-1)} \sum_{b=1}^B \sum_{t=2}^H \left(\Delta \hat{y}_t^{(b)} - \Delta y_{\text{target},t}^{(b)} \right)^2. \quad (2.10)$$

The total loss is then

$$\mathcal{L} = \mathcal{L}_{\text{fit}} + R_{\text{smooth}}^2 \mathcal{L}_{\text{dyn}}, \quad (2.11)$$

where $R_{\text{smooth}} \geq 0$ controls the strength of the regularization.

Interpretation. Unlike a generic smoothness constraint pushing derivatives toward zero, this term promotes *dynamics matching*: the predicted derivative is encouraged to follow the ground-truth derivative when the latter is significant, while being weakly constrained when the true derivative is near zero due to the weighting w_{dyn} .

Old smoothness formulation: derivative-to-zero augmentation

The previous training code implemented smoothness by augmenting the regression target and prediction arrays. The predicted derivative was computed as $\Delta\hat{y}$ and multiplied by a scalar R , after which the objective enforced this scaled derivative to be close to zero by concatenating it to the original regression target:

$$\hat{y}_{\text{aug}} := [\hat{y}, R \Delta\hat{y}], \quad (2.12)$$

$$y_{\text{aug}} := [y, 0], \quad (2.13)$$

and then applying the base criterion:

$$\mathcal{L}_{\text{old}} = \mathcal{C}(y_{\text{aug}}, \hat{y}_{\text{aug}}). \quad (2.14)$$

Key difference in the derivative definition

Both approaches use first differences of the predicted sequence; however, they differ in the *target* for those differences:

- **Old approach:** forces $R \Delta\hat{y}$ toward zero, encouraging constant outputs irrespective of the true dynamics.
- **New approach:** encourages $\Delta\hat{y}$ to match (a weighted version of) Δy , enabling changes when the true trajectory changes.

Advantages and disadvantages

New dynamics-matching penalty.

- **Advantage:** better physical plausibility, since the model is not penalized for changing output when the underlying system truly changes.
- **Advantage:** reduces oversmoothing artifacts (flat predictions) that can arise when enforcing $\Delta\hat{y} \approx 0$.
- **Disadvantage:** introduces additional hyperparameters (threshold τ and regularization strength R_{smooth}) and can amplify the impact of noisy derivatives at high sampling rates.

Old derivative-to-zero formulation.

- **Advantage:** simple regularizer that can reduce high-frequency jitter and stabilize training early.
- **Disadvantage:** structurally biases the solution toward constant trajectories, which can hurt performance when the motor speed profile contains genuine accelerations/decelerations.

Implementation note. In the new formulation, the smoothness penalty is explicitly additive (Eq. (2.11)), decoupled from the regression criterion $\mathcal{C}$. This simplifies ablation studies: $\mathcal{L}_{\text{fit}}$ can be changed (e.g., MSE vs time-weighted MSE) without modifying the definition of the smoothness term.

Chapter 3

Transformer Module Design

3.1 Activation Function in the Feed-Forward Network

Each Transformer block includes a position-wise feed-forward network (FFN) that performs non-linear feature mixing after the self-attention operation. In the implemented architecture, this module follows the standard two-layer structure:

$$\mathbf{x} \mapsto \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}), \quad (3.1)$$

where $\sigma(\cdot)$ is an element-wise activation function.

Initial choice: GELU

The initial implementation adopted the Gaussian Error Linear Unit (GELU) activation, defined as

$$\text{GELU}(x) = x \Phi(x), \quad (3.2)$$

where $\Phi(\cdot)$ denotes the cumulative distribution function of a standard normal variable. GELU is the default activation used in many Transformer architectures, including the original GPT models, and is known to provide smooth, non-linear gating of activations.

Adopted alternative: SiLU

In the present work, the activation function was generalized and the Sigmoid Linear Unit (SiLU), also known as the Swish activation, was preferred:

$$\text{SiLU}(x) = x \cdot \sigma(x), \quad (3.3)$$

where $\sigma(\cdot)$ denotes the logistic sigmoid.

The feed-forward network is therefore implemented as:

$$\text{FFN}(\mathbf{x}) = \text{Dropout}(\mathbf{W}_2 \text{SiLU}(\mathbf{W}_1 \mathbf{x})). \quad (3.4)$$

Motivation and discussion

The choice of SiLU over GELU is motivated by the following considerations:

- **Smooth gradients:** SiLU is continuously differentiable and exhibits non-zero gradients for negative inputs, which can improve gradient flow in deep networks.

- **Stable behavior for regression:** In contrast to ReLU-based activations, SiLU avoids hard saturation and dead units, which is beneficial in regression-oriented tasks such as continuous motor speed prediction.
- **Empirical robustness:** SiLU has shown favorable empirical performance in several modern architectures, particularly in settings where smooth input-output mappings are desired.

While GELU remains a strong and widely adopted default, the use of SiLU introduces a slightly less aggressive gating mechanism, which is better aligned with the continuous and physically smooth nature of the modeled dynamics. For this reason, SiLU was selected as the preferred activation function in the Transformer feed-forward blocks, while retaining the option to switch activations through configuration for ablation and comparison purposes.

Chapter 4

Proposed Improvements and Extensions

This chapter discusses additional improvements to the learning pipeline that were identified after defining the new regression logic, smoothness regularization, and training process. These improvements primarily address training–inference mismatch, model expressiveness, and evaluation robustness, rather than introducing new architectural components.

4.1 Mitigating Training–Inference Mismatch

The autoregressive training procedure feeds the model with its own previous predictions rather than ground-truth outputs. While this teacher-free setup improves robustness at inference time, it also makes optimization more challenging, especially in early training stages when prediction errors are large and compound rapidly.

A common mitigation strategy is *scheduled sampling*, where the model is intermittently conditioned on ground-truth outputs during training. Formally, at each time step t , the injected value is defined as

$$\tilde{y}_{t-1} = \begin{cases} y_{t-1}, & \text{with probability } p(t), \\ \hat{y}_{t-1}, & \text{with probability } 1 - p(t), \end{cases} \quad (4.1)$$

where $p(t)$ is a schedule that decreases over training epochs.

Motivation. Scheduled sampling reduces gradient variance and stabilizes early optimization, while still converging toward a fully autoregressive regime. In practice, this can lead to improved long-horizon predictions without sacrificing training stability.

Status. In the present implementation, fully teacher-free autoregression is used. Scheduled sampling is identified as a natural extension to further improve convergence behavior.

4.2 Model-Level Enhancements

Relative Temporal Encoding

The current Transformer implementation uses absolute positional embeddings. While effective, absolute encodings implicitly bind dynamics to fixed time indices within the window. An alternative is to adopt relative positional encodings, which emphasize temporal differences rather than absolute position.

Expected benefit. Relative encodings can improve generalization across windows of different lengths or across shifted temporal contexts, which is particularly relevant for dynamical systems where relative timing matters more than absolute index.

Predicting Increments Instead of Absolute Values

An additional model-level improvement consists in predicting output increments rather than absolute values:

$$\Delta\hat{y}_t = f_\theta(\cdot), \quad \hat{y}_t = \hat{y}_{t-1} + \Delta\hat{y}_t. \quad (4.2)$$

Motivation. Incremental prediction often improves numerical stability and reduces drift in autoregressive rollouts, as the model focuses on local dynamics rather than reconstructing absolute magnitudes at each step. This formulation also aligns naturally with derivative-based regularization.

4.3 Evaluation Strategy and Test-Time Monitoring

A separate test dataset split is constructed during data preprocessing. Evaluation on the test split is performed outside the training loop, using a dedicated evaluation function that mirrors the inference-time rollout.

Role of the test step

Importantly, incorporating a test step during training does *not* influence the learned model parameters. No gradients are computed, and no optimizer updates are performed based on test-set performance.

What changes. Adding periodic test evaluation during training enables:

- tracking generalization error over training epochs;
- detecting overfitting or performance degradation early;
- comparing regression modes and regularization strategies under identical evaluation conditions;
- analyzing long-horizon rollout behavior on unseen trajectories.

What does not change. The test step does *not*:

- modify network weights or biases;
- alter the training objective;
- improve predictions by itself.

Why this still matters

Although test evaluation does not directly affect optimization, it plays a critical role in experimental correctness. Without systematic test-time monitoring, improvements in training loss may not translate to improved predictive performance under inference conditions, particularly for autoregressive models where small errors can accumulate over time.

Summary. The inclusion of structured evaluation and monitoring improves the reliability and interpretability of experimental results, even though it does not directly intervene in the parameter update process.